Covered: Browser Agents Cannot Tell a Clickable Element From an Occluded One

Bhargav Pedapudi
Independent Researcher
San Francisco Bay Area, CA, USA
ORCID: 0009-0002-8523-8415
pedapudibhargav@gmail.com

***Abstract*—A browser agent is given a list of named interactive elements. When a consent dialog covers the page, that list contains the covered elements exactly as it contains the reachable ones. The agent selects the correct target, the click is intercepted, and nothing in its observation explains the failure, so it repeats the identical action until its step budget is exhausted. We audit the two tree serialisations agents actually receive and find that neither expresses occlusion on any of eight live sites carrying 25 to 141 covered elements, and that the seven accessibility-tree properties Chrome defines for this purpose are emitted on no node in any frame. We then evaluate the one deployed detector: a paint-order rule recovers 0.84 to 0.95 of covered elements when the blocker is in the main document and 0.00 to 0.17 when it is inside a cross-origin iframe, missing 146 of 373 covered elements overall with zero false positives, because paint order is computed per document while the compositor is not. Hit testing closes both gaps in 3 ms. Labelling occlusion in the observation moves task success from 4/26 to 19/26 across eleven live sites (McNemar exact two-sided $p = 0.00006$; seven sites better, none worse, $p = 0.0156$) at fewer observation tokens, replicates on a second model, and reproduces on a deterministic replay benchmark we release. A four-arm ablation separates diagnosis from detection: a loop detector given a generic "that failed, try something else" hint scores 4/17 against a flat baseline's 3/17, so what recovers the agent is being told *what* is wrong, not that something is. We report five retracted claims of our own, including a compression result that reversed sign under a proper paired test.**

***Index Terms*—web agents, large language models, browser automation, accessibility tree, occlusion, Chrome DevTools Protocol, reproducible benchmarks**

I. Introduction

An LLM browser agent acts through an observation of the page. In current practice that observation is a flat list of interactive elements, each with a role, an accessible name and a reference the agent can act on. The list is produced either from the Chrome DevTools Protocol accessibility tree or from Playwright's snapshot, and both are in wide deployment.

Neither can say that an element cannot be reached.

The consequence is a specific and reproducible failure. On a page where a consent dialog intercepts pointer events, the agent is handed a list in which the covered navigation links appear with correct names and correct references. It selects the right one. The click times out. The observation on the next step is unchanged, so the agent selects the same element again:

```
click('27') -> err:TimeoutError
click('27') -> err:TimeoutError
click('27') -> err:TimeoutError
... six identical attempts, task failed
```

The agent did not choose badly. It was given a representation in which the distinction it needed does not exist.

This paper makes that argument by measurement. Section III audits what the deployed serialisations contain. Section IV evaluates the one shipped occlusion detector and shows that its method is blind to the most common kind of full-page overlay. Section V measures what supplying the missing fact is worth, and separates it from the cheaper alternative of detecting failure. Section VI decomposes the cost of a step. Section VII describes the deterministic benchmark we release. Section VIII reports what we got wrong.

II. Related Work

Agents fail on consent banners, and it is documented without a cause. BrowserArena [1] names pop-up banner removal as one of three recurring open-web failure modes. Over 80 tasks on a single news site it reports the banner detected in 100.00% of Claude-3.7 runs and closed in only 7.5%; Llama-4 detected 98.75% and closed 17.72%; o4-mini closed 17.81% and Gemini 2.5 closed 4.65% (Appendix K, Table 4). The failure is quantified; no mechanism is given beyond the name of the task.

Repeated ineffective actions are a studied failure class. MIRAGE-Bench [2] constructs settings in which an agent is persistently executing the same incorrect action, probing at the third and sixth repetition (Appendix A.6.1). A human audit of WebArena-Lite trajectories [3] reviews 102 failed GPT-5.5 runs and reports frequent scrolling loops. Neither attributes a loop to occlusion.

The premise of this paper has been stated once, in passing. MIRAGE-Bench's pop-up setting appends pop-up elements to the end of the accessibility tree, and its Figure 8 caption observes that the pop-up "visually occludes part of the interface, but does not remove underlying components from the accessibility tree" [2]. That is the gap we measure. It appears in a figure caption; it is not quantified, not fixed and not ablated.

Telling an agent to ignore a pop-up does not work. Zhang et al. [4] achieve an 86% attack success rate with adversarial

pop-ups, decreasing task success by 47%, and report that basic defences such as asking the agent to ignore pop-ups are ineffective. Their setting is adversarial rather than benign consent, and they do not compare against an observation-side remedy. Our Section V supplies that comparison for the benign case.

Overlays measurably reduce agent success in industry testing. A practitioner study [5] reports agent success rising from approximately 50% to approximately 64% on a full-screen consent banner between two measurement rounds, and from approximately 32% to approximately 41% on a transparent `opacity:0` overlay that swallows clicks. No mechanism is offered.

Occlusion is detectable, and one deployed system detects it. Browser Use marks a node `ignored_by_paint_order` when its rectangle lies inside the union of rectangles painted above it, using CDP paint order [6]. To our knowledge this has never been evaluated. Section IV evaluates it.

Observation-format ablations vary modality, not reachability. Published comparisons set screenshots against accessibility trees, or set-of-marks against text. Work on reducing the observation space includes region-based reorganisation, which reports observation tokens falling from 6,437 to 3,671 and success improving by 2.3 percentage points [8]. We are not aware of any work that varies whether the agent is told which elements it can reach.

III. What the Deployed Formats Contain

A. Method

We load eight sites that present a blocking consent dialog and serialise each page three ways: the CDP `Accessibility.getFullAXTree` call on the top document; the same call issued per frame and concatenated, which is the approach BrowserGym takes via `Page.getFrameTree` [7]; and Playwright's `aria_snapshot(mode="ai")`, which is what Playwright MCP serves to an agent. For each serialisation we ask two questions: does it contain the control that dismisses the dialog, and does it state anywhere that elements are covered.

For the second question we search each serialisation for any of the terms a format would have to use to express occlusion, and we additionally enumerate the accessibility-tree properties Chrome's protocol defines for this purpose: `inertElement`, `inertSubtree`, `activeModalDialog`, `activeAriaModalDialog`, `ariaHiddenSubtree`, `notRendered` and `notVisible`.

B. Result

Table I gives the result. Two findings separate.

First, frame blindness belongs to one integration pattern, not to web agents in general. A top-document-only `getFullAXTree` misses the dismiss control on 4 of 4 sites where the consent panel renders inside a cross-origin iframe. Issued per frame and merged, the same call finds it on 4 of 4. So does Playwright's AI-mode snapshot, which emits the control with a frame-qualified reference, for example `button "Accept and Continue" [ref=f4e22]`. A naive integration is blind here; the tools are not.

TABLE I
What each deployed serialisation contains, on eight live sites carrying 25–141 covered elements each.

Serialisation	Dismiss control	Occlusion stated
`getFullAXTree`, top document	0 / 4	**0 / 8**
`getFullAXTree`, merged per frame	4 / 4	**0 / 8**
`aria_snapshot(mode="ai")`	4 / 4	**0 / 8**
This work (hit tested)	4 / 4	8 / 8

Second, and this is the gap: **occlusion is expressed by none of the three, on any of the eight sites.** Between 25 and 141 interactive elements per page are covered and unclickable, and all three serialisations list them exactly as they list reachable elements. Chrome's own vocabulary for this is unused — none of the seven properties above was emitted on any node, in any frame, on any site.

One apparent exception was a false positive worth recording: a keyword search matched the word "interceptor" in a news headline. We mention it because the measurement is a search over text and such matches must be checked rather than counted.

IV. Paint Order Misses the Overlays That Matter

A. Method

Ground truth for "would a click land here" is `document.elementFromPoint`, which reports what the compositor hits. For each visible interactive element we hit-test its centre and treat the element as covered when the returned node is neither the element nor an ancestor nor a descendant of it.

Against that we evaluate the paint-order rule. We read real paint order from `DOMSnapshot.captureSnapshot` with `includePaintOrder`, rather than approximating it from document order, and mark an element covered when its rectangle lies entirely inside the rectangle of a node painted above it, with contributors below 0.8 opacity excluded as in the deployed implementation [6].

An earlier version of this comparison approximated paint order from document order and produced a different and misleading answer; we report the CDP-based figures only.

B. Result

Table II shows 146 of 373 covered elements missed, with *zero* false positives: the rule is conservative rather than noisy.

The failures are not spread evenly. Recall is near zero on four sites and near one on four others, and the division is mechanistic. We classified each page's blocking element and found, with no exceptions in eight sites, that the four low-recall sites are exactly those whose consent dialog renders its panel inside a cross-origin iframe.

TABLE II
PAINT-ORDER RECALL AGAINST HIT TESTING, EIGHT LIVE SITES. THE SPLIT IS BY WHAT IS BLOCKING, AND IT HOLDS 8 OF 8.

Site	Covered	Found	Recall
Blocker inside a cross-origin iframe			
BBC	33	0	**0.00**
Financial Times	30	0	**0.00**
Spiegel	27	1	**0.04**
Sky Sports	42	7	**0.17**
Blocker in the main document			
Le Monde	25	21	0.84
Euronews	141	127	0.90
CNN	36	34	0.94
Irish Times	39	37	0.95
Total	373	227	0.61

The reason is that **paint order is computed per document**. When the panel lives in a cross-origin iframe, the opaque rectangle that covers the page is not a node in the top document's snapshot. Nothing in that document is painted above the links, so nothing is marked covered — while the compositor places the frame on top and sends the click to it. A hit test does not have this problem, because `elementFromPoint` reports what the compositor actually hit, across document boundaries, and costs 3 ms for a whole page.

This matters because the iframe case is the common case rather than the exotic one. The major consent-management platforms render their panel in an iframe. In an independent sweep of 42 further candidate sites, most of the sites we rejected were rejected for exactly this reason — blocked, with no dismiss control reachable in the top document — including pages with 59, 56, 181 and 252 covered elements.

V. DOES LABELLING OCCLUSION HELP?

A. Setup

We construct 26 tasks on 11 live sites. Each task's target is a navigation link that sits behind a consent dialog, and success is a URL check, so nothing is judged by a model. The task set was fixed and committed before the run.

All arms share the model, the action space and a six-step budget. Each task gets a fresh browser context rather than cleared cookies, because `clear_cookies()` leaves `localStorage` intact and consent state lives there; an earlier run without this inflated the effect when one arm dismissed a dialog and its next tasks on the same site loaded with no dialog at all.

The arms differ only in the observation:

- **A, flat.** The element list, as current practice produces it.
- **B, structured.** The same list, with the overlay named, the covered elements collapsed into a line of references, and the overlay's own controls listed.
- **B2, structured with four failure-driven additions** (Section V-D).
- **C, flat with a specific hint** injected when an action repeats: the hint names overlays and lists dismiss verbs.

TABLE III
OCCLUSION LABELLING, 26 TASKS ON 11 LIVE SITES.

	`gpt-5-mini`	`gpt-5.4-mini`
A, flat	4 / 26	4 / 26
B, structured	16 / 26	**16 / 26**
B2, with fixes	**19 / 26**	—
paired vs. flat	$+15/-0$	$+14/-2$
McNemar exact two-sided	$\mathbf{p = 0.00006}$	$\mathbf{p = 0.00418}$
by site	7 better, 0 worse	6 better, 1 worse
sign test	$p = 0.0156$	$p = 0.125$
steps per task	5.23 → **3.38**	5.23 → 3.77
observation tokens	3,008 → **2,887**	—

TABLE IV
FOUR ARMS, 17 TASKS. DETECTING THE FAILURE IS NOT WHAT HELPS.

Arm	Solved	vs. flat
A, flat	3 / 17	—
D, flat + *generic* hint	4 / 17	$+1/-0$, $p = 1.00$
B, occlusion labelled	12 / 17	$+10/-1$, $p = 0.0117$
C, flat + *specific* hint	13 / 17	$+10/-0$, $p = 0.0020$

- **D, flat with a generic hint**: "your last action was repeated and had no effect. Something is preventing it from working. Try a different approach."

B. Labelling works, on two models

Table III gives the result. Eleven of the flipped tasks are the same tasks under both models, which argues that the effect belongs to the page rather than to one model's idiosyncrasies.

We report the site-level sign test alongside the task-level test because 26 tasks on 11 sites are not 26 independent trials: tasks on one site share an overlay. Counting sites as the unit is the conservative reading, and it remains significant for the first model.

The structured observation is *smaller*. Collapsing 33 covered elements into a line of references costs fewer tokens than listing all 33 as though they worked, so the accuracy gain here does not trade against cost.

The two control sites, where the dialog is present but does not obstruct the navigation, tie in both arms — which is what a control should do, and which rules out a general benefit from the richer format.

C. Diagnosis, not detection

A loop detector is the obvious cheaper fix: the flat arm emits the same timing-out click six times, so telling it "that repeated and failed" should suffice. It does not.

Table IV separates the two explanations. Given a hint that names the failure mode and lists the remedy, the detector matches the structured observation. Given a hint that says only that something is wrong, it is worth one task out of seventeen and is statistically indistinguishable from silence.

The traces show the detector working and not helping:

```
click('27')  -> err:TimeoutError   correct target
click('27')  -> err:TimeoutError   repeat detected
```

```
click('525') -> err:TimeoutError   varies, as instructed
click('27')  -> err:TimeoutError
click('21')  -> err:TimeoutError
click('593') -> err:TimeoutError   budget gone
```

The agent obeys. It stops repeating and starts varying, and every variation times out too, because everything it can see is behind the same overlay and nothing in its observation mentions one. Varying blindly over a covered page is not a recovery strategy.

Arm C's hint is an oracle: hand-written for the failure it was evaluated on, correct for exactly one failure mode, and selectable only by a mechanism that would already have to know what is wrong. It also pays a fixed tax. On the tasks both C and B solve, B takes 2.33 steps and C takes 3.50, because a detector cannot fire until the agent has already spent the actions that prove the failure.

So the effect is carried by the diagnosis, not by the detection. Not "you are looping" but "you are covered".

D. Four fixes from four failures

We examined every task that failed in the structured arm and derived one change from each cause: an imperative first line naming the control to activate; a ranking over the overlay's controls; a parameter-free obstruction test, which suppresses the announcement when the overlay covers nothing in a `navigation` or `main` landmark; and a note when the dismiss control is in an iframe and therefore absent from the listing.

Together these move 16/26 to 19/26 and cut steps by 19%. Against flat the improved arm is $+15/-0$, $p = 0.00006$, and by site 7 better and 0 worse, $p = 0.0156$. **Against plain structure it is $+4/-1$, $p = 0.375$ — not significant.** The failure analysis improved the system; at $n = 26$ the improvement itself is inside noise, and we state it as such.

The control ranking is correct on 2 of 6 sites using only generic structural signals (a button rather than a link, `autofocus`, a declared default action, area) and on 5 of 6 once an accept-word and trap-word list is added. The vocabulary is doing the work, so we present the ranking as an engineering default with a measured contribution rather than as a structural result.

E. Wording decides whether the agent keeps trying

One of the four fixes caused a regression, and it is the most transferable result in this section.

On a site whose blocker is an advertisement iframe rather than a consent dialog, the fourth fix emitted: "No control in this document closes it." The agent read a statement of absence as a statement of impossibility. It stopped exploring and clicked one element five times, where the same arm without the line had succeeded in four steps.

Reworded to state the same fact without closing the door — "its controls are not listed here; the page behind it may still respond" — one of the two affected tasks recovers.

The channel that rescues an agent by naming what is covered can disable it by implying nothing can be done. An observation format is not a neutral description of a page; its phrasing changes whether the agent continues to search.

TABLE V
WHERE A STEP'S WALL CLOCK GOES, INSTRUMENTED OVER 193 STEPS.

Phase	Mean	Median	Share
Observation extraction	44 ms	15 ms	2.4%
Model call	887 ms	833 ms	47.7%
Browser action	928 ms	104 ms	49.9%
Step total	1,859 ms		

VI. The Cost of Not Knowing

Table V decomposes a step. Extraction is 2.4% of it, so the 3 ms hit test is not a cost worth optimising. The model call and the browser action are roughly half each, which is consistent with a profiling study reporting median delays of 4.7 s for LLM inference and 6.6 s for browser action time [9].

The mean browser action time hides the structure that matters:

Action outcome	Mean	Median
Lands ($n = 148$)	215 ms	**87 ms**
Intercepted ($n = 45$)	3,390 ms	**4,004 ms**

An intercepted click costs four seconds, 46 times one that lands. A flat arm that spends six steps on a covered page therefore burns roughly 24 seconds in dead browser timeouts alone, on top of six model calls. Occlusion blindness costs wall clock twice: in extra round trips, and in a four-second stall on each blocked click.

Observation size is not the lever here. Over the range agents work in we measured a 72× prompt range costing 26% in latency; an earlier figure of 11% over a 32× range was an artifact of prompt caching, since the sweep had sent five identical prompts per size.

VII. A Deterministic Benchmark

Every number above was measured on live sites, and that is the weakest property of the study. Five of eleven sites showed no overlay when re-probed minutes after reporting 17 to 182 covered elements. Consent dialogs vary with geography, experiment assignment and prior state, and nobody else can rerun a measurement taken this way.

We therefore record each site's network traffic to HAR with the overlay present and serve a fresh browser context entirely from that recording, with a request the recording does not contain failing rather than reaching the network. A recording is used only if it reproduces the condition under study: we require the overlay to be present on replay and the covered-element count to be within 35% of the live figure, and the experiment runs only the sites that pass.

Ten of eleven recordings reproduce their overlay, nine with an identical covered-element count, including all four sites that had been unstable live. The instability was in the network, not in the page.

The headline reproduces deterministically: 2/24 to 11/24, $+9/-0$, McNemar exact two-sided $p = 0.00391$, at 2,925 to 2,525 observation tokens. Magnitude is lower than live and the site-level sign test falls to 4 better and 0 worse, $p = 0.125$. Replay is also slower — a step averages 3,141 ms against 1,859 ms, with browser action time 2,300 ms against 928 ms — so the decomposition in Section VI belongs to the live runs and the two must not be pooled.

Building the fixture produced one instructive defect. The first recording captured only each site's landing page, and the replayed experiment returned 0/17 in both arms, which reads as a clean refutation of the entire result. It was the fixture: the success condition is a URL change, the destination pages were never recorded, and an unrecorded navigation aborts, so an agent could dismiss the overlay, click the correct link and still fail. Three things in the data indicated this before any conclusion was drawn — the improved arm's observation was 2,011 tokens against the flat arm's 3,052, so the overlay was being handled correctly; every task used the full budget in both arms; and the recording contained exactly one document for the site under test. The recorder now visits every target path during capture.

We release the code, the recordings and the raw results for every number in this paper.

VIII. Negative Results and Retractions

We report these because they are what makes the positive result credible, and because each was found by testing our own claim.

Filtering the observation is harmful, not neutral. We previously reported compression as an empty direction on the basis of 14 tasks with no paired test. At 40 paired tasks on a component benchmark, filtering the observation to reachable, named, actionable elements cuts it by 80% (median 1,607 to 197.5 characters) and costs 36/40 to 30/40, with 0 gained and 6 lost, McNemar $p = 0.03125$. Five of the six lost tasks go from one or two steps to exhausting the budget, and they are a drawer, a collapsible disclosure, a notification centre, a custom select and a text input — controls whose contents are hidden until revealed, which is exactly the set a reachability filter removes.

An attribution of that loss, which failed. The appealing reading is a symmetry with Section V: labelling occlusion helps, deleting it hurts, and the deployed practice of dropping covered nodes is thereby indicted. One ablation refuted it. Keeping covered elements in the filtered observation, with the other filters unchanged, recovered 0 of the 6 lost tasks and scored slightly worse again (28/40). The harm belongs to the filtered serialisation as a whole; which component carries it is unlocalised.

Cross-frame traversal is not a contribution. We implemented frame traversal with frame-qualified references and reported it as closing a gap no page observation could reach. Playwright's AI-mode snapshot already descends into cross-origin iframes and already uses that reference scheme, and BrowserGym already merges per-frame trees. The claim is withdrawn; what survives is the narrower statement in Section III about a top-document-only integration.

Occlusion detection is not absent from deployed systems. We claimed it was. Browser Use ships it. Section IV is the result of checking.

A prevalence citation we cut. A figure we had attributed to a consent measurement study is not that study's statistic, and we removed the claim rather than soften it. We are not aware of a prevalence figure for blocking consent dialogs that we can stand behind, and we make no prevalence claim.

IX. Limitations

The experiments cover one task family — navigation, with success by URL check — and one overlay type, the consent dialog. Autocomplete menus, sticky headers, interstitials and transparent click-swallowing overlays are not measured. Twenty-six tasks on eleven sites is a small study, and the improvement of the failure-driven additions over plain structure is not significant at that size. Two models, from one family, are not model independence. The paint-order comparison reimplements the deployed rule from its source rather than running that system end to end, which could be unfair to it in ways a reimplementation hides. One site reproduces its overlay on replay yet collapses from 4/4 live to 0/4 replayed in both arms, and we have not diagnosed it.

X. Conclusion

The observation formats browser agents are given cannot express the difference between an element that can be clicked and one that cannot. The fact is available from the DOM for 3 ms and at negative token cost, the accessibility protocol already defines a vocabulary for it that goes unused, and the one deployed detector uses a per-document method blind to the overlays that occur most. Supplying the fact moves success from 4/26 to 19/26 across eleven sites and reproduces deterministically. What recovers the agent is being told what is wrong rather than that something is: a loop detector with a generic hint is worth one task in seventeen. And the same channel can disable the agent if it states an absence as an impossibility.

Artifact Availability

Code, the HAR recordings, and the raw results behind every number are at https://github.com/pedapudibhargav/glance-occlusion.

References

[1] S. Anupam, D. Brown, S. Li, E. Wong, H. Hassani, and O. Bastani, "BrowserArena: Evaluating LLM agents on real-world web navigation tasks," arXiv:2510.02418, 2025.

[2] W. Zhang, Y. Sun, P. Huang, J. Pu, H. Lin, and D. Song, "MIRAGE-Bench: LLM agent is hallucinating and where to find them," arXiv:2507.21017, 2025.

[3] C. Gan, Z. He, Y. Tsujii, K. Kobayashi, H. Itoh, and K. Funakoshi, "Auditing web agent evaluation on WebArena-Lite: Human review of outcomes and trajectories," arXiv:2610.01491, 2026.

[4] Y. Zhang, T. Yu, and D. Yang, "Attacking vision-language computer agents via pop-ups," in *Proc. ACL*, 2025. arXiv:2411.02391.

[5] Merj, "How AI agents use websites, where they fail, and what to fix," 2026. [Online]. Available: https://merj.com/blog/agent-interaction-research
[6] Browser Use, "`browser_use/dom/serializer/paint_order.py`," source code. [Online]. Available: https://github.com/browser-use/browser-use
[7] ServiceNow Research, "`browsergym/core/observation.py`," source code. [Online]. Available: https://github.com/ServiceNow/BrowserGym
[8] D. Kwon and D. Lee, "Rethinking observation space granularity for web agents," arXiv:2605.07134, 2026.
[9] M. Wong, K. Hsieh, S. Nath, and R. Netravali, "Speculative execution for fast and efficient web agents," arXiv:2605.16565, 2026.